DBMS Lab 08 - Isolation Level in Transaction Management

E-Commerce Order & Inventory Management - A Case Study

SRN	PES1UG24AM387
Name	Pranaav
MySQL account	pes1ug24am387_pranaav@localhost
MySQL version	8.0 (InnoDB)
Client prompt	prompt \u@\h [\d]>_

Session 1 runs Transaction A and Session 2 runs Transaction B. The two run in separate MySQL client windows, both logged in as **pes1ug24am387_pranaav**. Each screenshot is labelled with its question number and session.



Setup - prompt set in both sessions

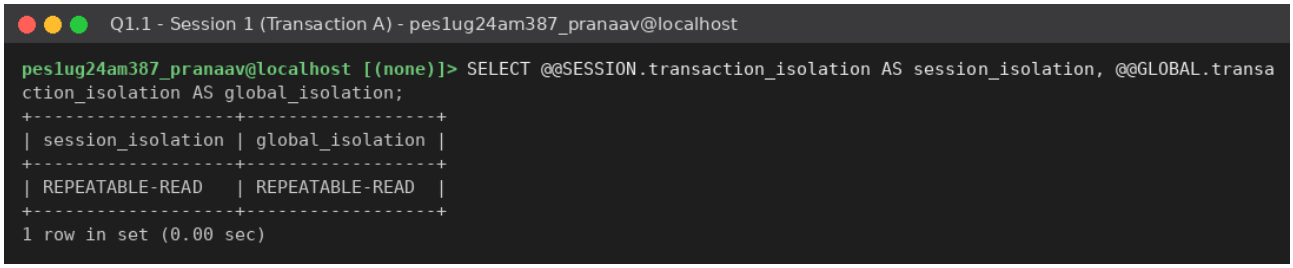
Task 1: Check MySQL Isolation Level

Question 1.1

Display the transaction isolation level that applies to your current session, and the global (server-wide) default isolation level that every new session starts with.

SQL executed - Session 1 (Transaction A):

```
SELECT @@SESSION.transaction_isolation AS session_isolation, @@GLOBAL.transaction_isolation AS global_isolation;
```



Screenshot: Question 1.1 - Session 1 (Transaction A)

Observation: Both the session and the global isolation level are REPEATABLE-READ, which is the InnoDB default.

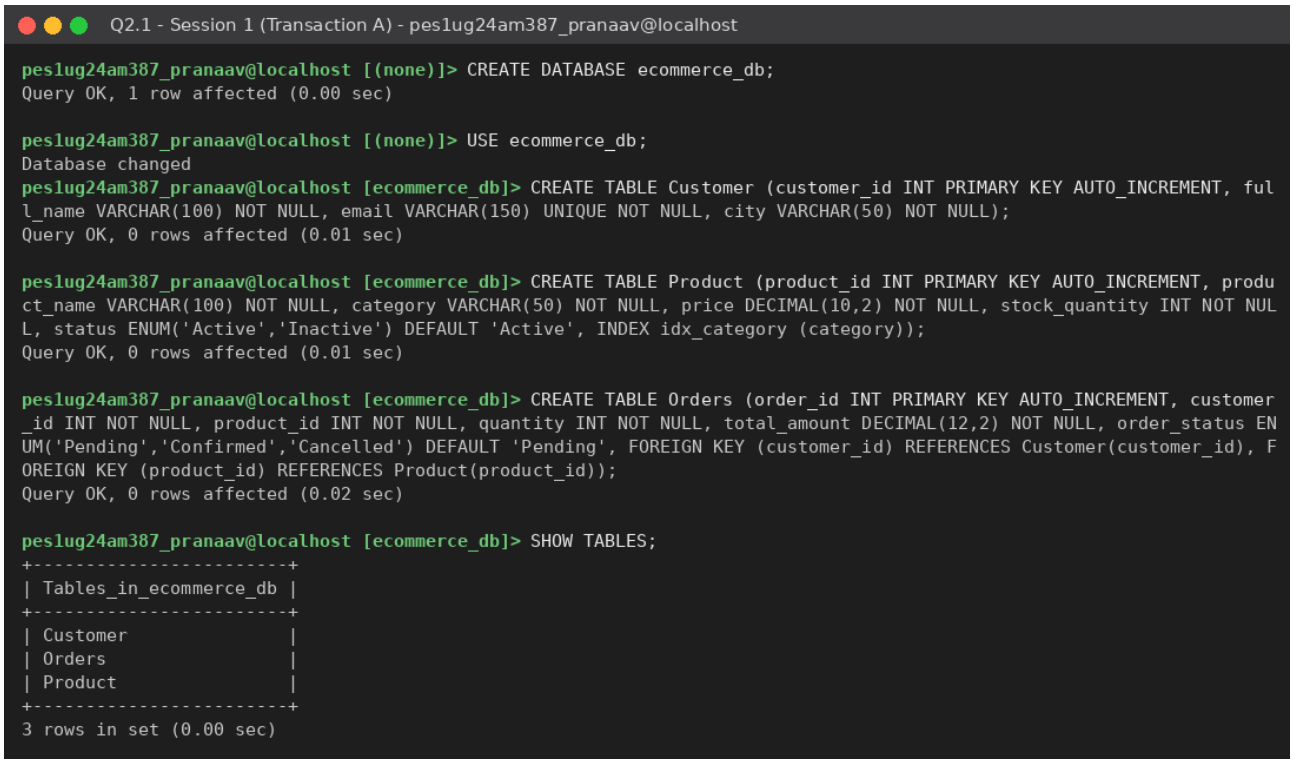
Task 2: Database Setup and Sample Data

Question 2.1

Create the ecommerce_db database, make it the active database, and create the Customer, Product and Orders tables exactly as supplied with the case study.

SQL executed - Session 1 (Transaction A):

```
CREATE DATABASE ecommerce_db;
USE ecommerce_db;
CREATE TABLE Customer (customer_id INT PRIMARY KEY AUTO_INCREMENT, full_name VARCHAR(100) NOT NULL, email
    VARCHAR(150) UNIQUE NOT NULL, city VARCHAR(50) NOT NULL);
CREATE TABLE Product (product_id INT PRIMARY KEY AUTO_INCREMENT, product_name VARCHAR(100) NOT NULL, category
    VARCHAR(50) NOT NULL, price DECIMAL(10,2) NOT NULL, stock_quantity INT NOT NULL, status
    ENUM('Active','Inactive') DEFAULT 'Active', INDEX idx_category (category));
CREATE TABLE Orders (order_id INT PRIMARY KEY AUTO_INCREMENT, customer_id INT NOT NULL, product_id INT NOT NULL,
    quantity INT NOT NULL, total_amount DECIMAL(12,2) NOT NULL, order_status
    ENUM('Pending','Confirmed','Cancelled') DEFAULT 'Pending', FOREIGN KEY (customer_id) REFERENCES
    Customer(customer_id), FOREIGN KEY (product_id) REFERENCES Product(product_id));
SHOW TABLES;
```



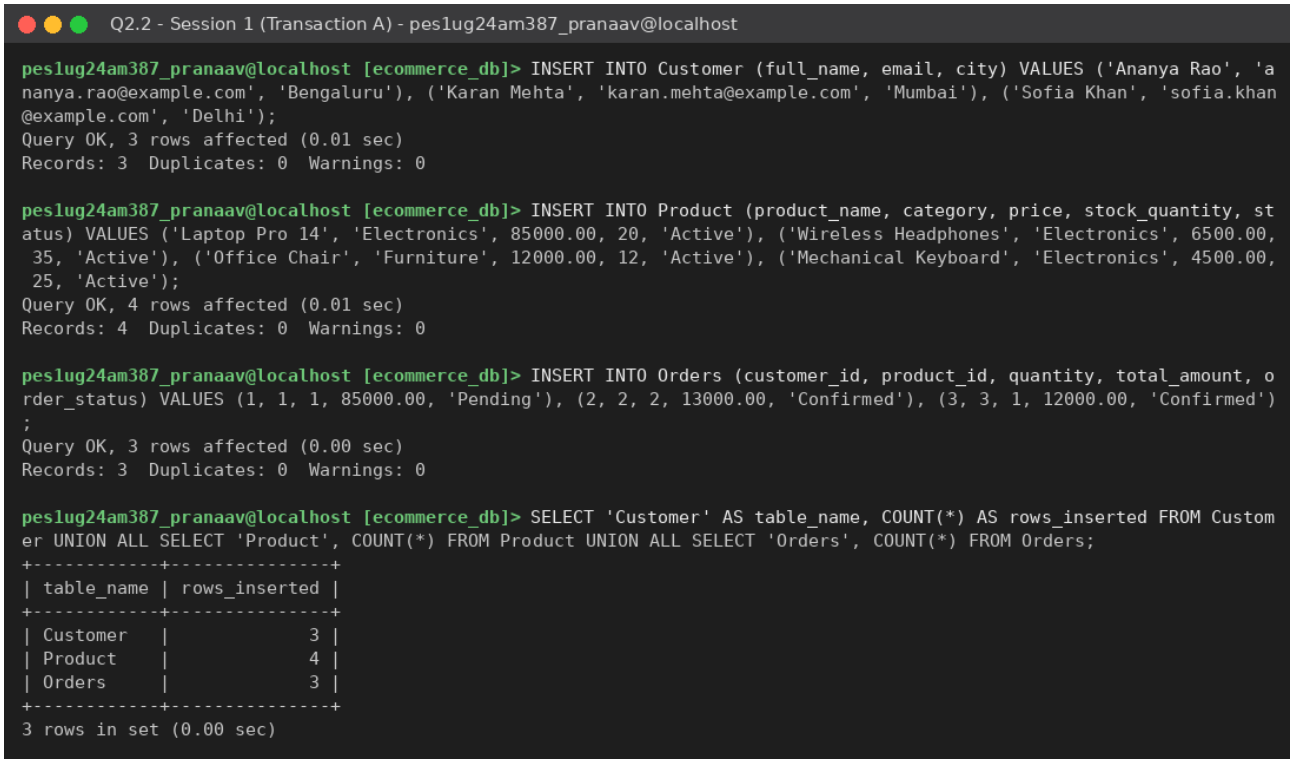
Screenshot: Question 2.1 - Session 1 (Transaction A)

Question 2.2

Insert the supplied customers, products and orders into the three tables and record the number of rows inserted into each table.

SQL executed - Session 1 (Transaction A):

```
INSERT INTO Customer (full_name, email, city) VALUES ('Ananya Rao', 'ananya.rao@example.com', 'Bengaluru'),
    ('Karan Mehta', 'karan.mehta@example.com', 'Mumbai'), ('Sofia Khan', 'sofia.khan@example.com', 'Delhi');
INSERT INTO Product (product_name, category, price, stock_quantity, status) VALUES ('Laptop Pro 14',
    'Electronics', 85000.00, 20, 'Active'), ('Wireless Headphones', 'Electronics', 6500.00, 35, 'Active'),
    ('Office Chair', 'Furniture', 12000.00, 12, 'Active'), ('Mechanical Keyboard', 'Electronics', 4500.00, 25,
    'Active');
INSERT INTO Orders (customer_id, product_id, quantity, total_amount, order_status) VALUES (1, 1, 1, 85000.00,
    'Pending'), (2, 2, 2, 13000.00, 'Confirmed'), (3, 3, 1, 12000.00, 'Confirmed');
SELECT 'Customer' AS table_name, COUNT(*) AS rows_inserted FROM Customer UNION ALL SELECT 'Product', COUNT(*)
    FROM Product UNION ALL SELECT 'Orders', COUNT(*) FROM Orders;
```



Screenshot: Question 2.2 - Session 1 (Transaction A)

Observation: Rows inserted: Customer = 3, Product = 4, Orders = 3.

Question 2.3

Display the complete contents of the Customer, Product and Orders tables to verify the initial data.

SQL executed - Session 1 (Transaction A):

```
SELECT * FROM Customer;
SELECT * FROM Product;
SELECT * FROM Orders;
```

Q2.3 - Session 1 (Transaction A) - pes1ug24am387_pranaav@localhost

```
pes1ug24am387_pranaav@localhost [ecommerce_db]> SELECT * FROM Customer;
+-------------+-------------+-------------------------+-----------+
| customer_id | full_name   | email                   | city      |
+-------------+-------------+-------------------------+-----------+
|           1 | Ananya Rao  | ananya.rao@example.com  | Bengaluru |
|           2 | Karan Mehta | karan.mehta@example.com | Mumbai    |
|           3 | Sofia Khan  | sofia.khan@example.com  | Delhi     |
+-------------+-------------+-------------------------+-----------+
3 rows in set (0.00 sec)

pes1ug24am387_pranaav@localhost [ecommerce_db]> SELECT * FROM Product;
+------------+---------------------+-------------+----------+----------------+--------+
| product_id | product_name        | category    | price    | stock_quantity | status |
+------------+---------------------+-------------+----------+----------------+--------+
|          1 | Laptop Pro 14       | Electronics | 85000.00 |             20 | Active |
|          2 | Wireless Headphones | Electronics |  6500.00 |             35 | Active |
|          3 | Office Chair        | Furniture   | 12000.00 |             12 | Active |
|          4 | Mechanical Keyboard | Electronics |  4500.00 |             25 | Active |
+------------+---------------------+-------------+----------+----------------+--------+
4 rows in set (0.00 sec)

pes1ug24am387_pranaav@localhost [ecommerce_db]> SELECT * FROM Orders;
+----------+-------------+------------+----------+--------------+--------------+
| order_id | customer_id | product_id | quantity | total_amount | order_status |
+----------+-------------+------------+----------+--------------+--------------+
|        1 |           1 |          1 |        1 |     85000.00 | Pending      |
|        2 |           2 |          2 |        2 |     13000.00 | Confirmed    |
|        3 |           3 |          3 |        1 |     12000.00 | Confirmed    |
+----------+-------------+------------+----------+--------------+--------------+
3 rows in set (0.00 sec)
```

Screenshot: Question 2.3 - Session 1 (Transaction A)

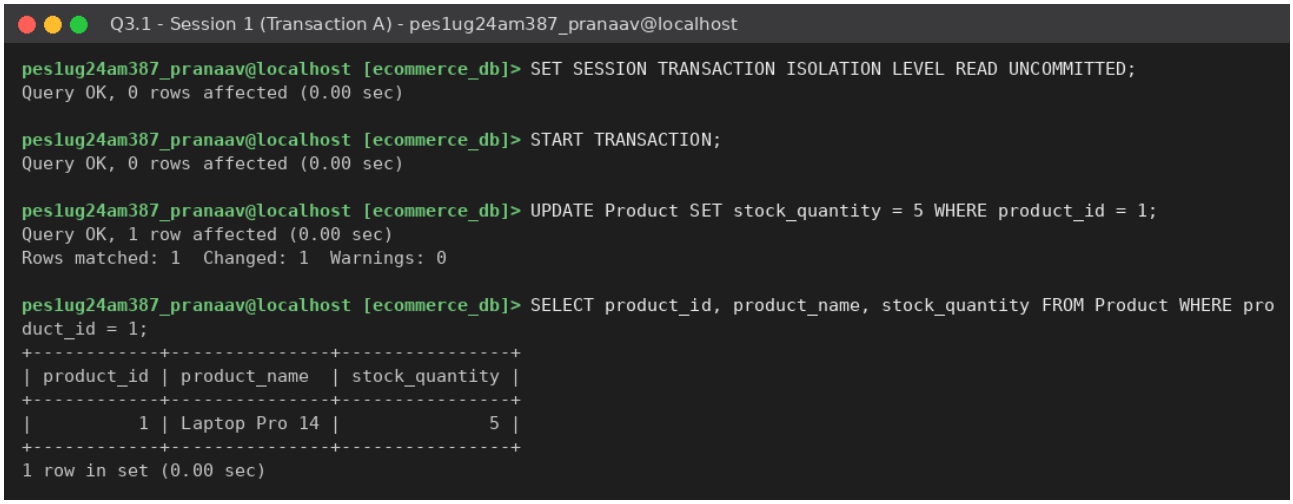
Task 3: READ UNCOMMITTED

Question 3.1

In Session 1 (Transaction A), set the isolation level to READ UNCOMMITTED, start a transaction and reduce the stock quantity of product 1 to 5. Do not issue COMMIT.

SQL executed - Session 1 (Transaction A):

```
SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
START TRANSACTION;
UPDATE Product SET stock_quantity = 5 WHERE product_id = 1;
SELECT product_id, product_name, stock_quantity FROM Product WHERE product_id = 1;
```



Screenshot: Question 3.1 - Session 1 (Transaction A)

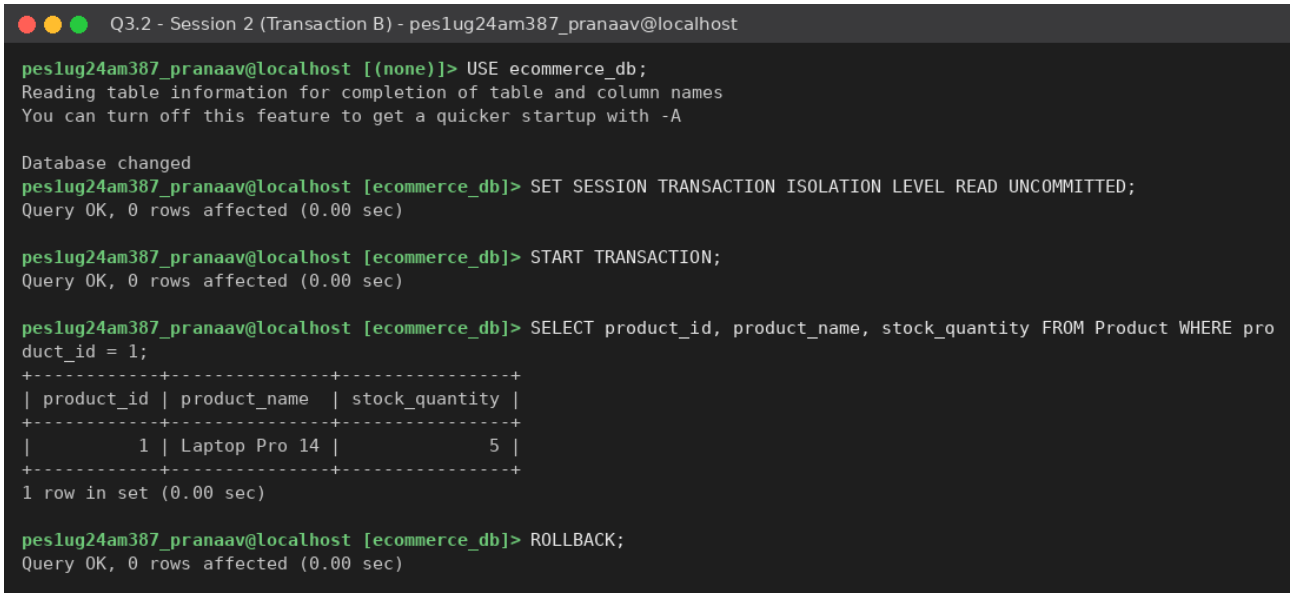
Observation: Transaction A sees stock_quantity = 5 for product 1. The change is still uncommitted.

Question 3.2

From a second session (Transaction B), set READ UNCOMMITTED, start a transaction and read the product_id, product_name and stock_quantity of product 1 while Transaction A is still uncommitted. Determine whether Transaction B can see the changed stock, then roll back both transactions.

SQL executed - Session 2 (Transaction B):

```
USE ecommerce_db;
SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
START TRANSACTION;
SELECT product_id, product_name, stock_quantity FROM Product WHERE product_id = 1;
ROLLBACK;
```

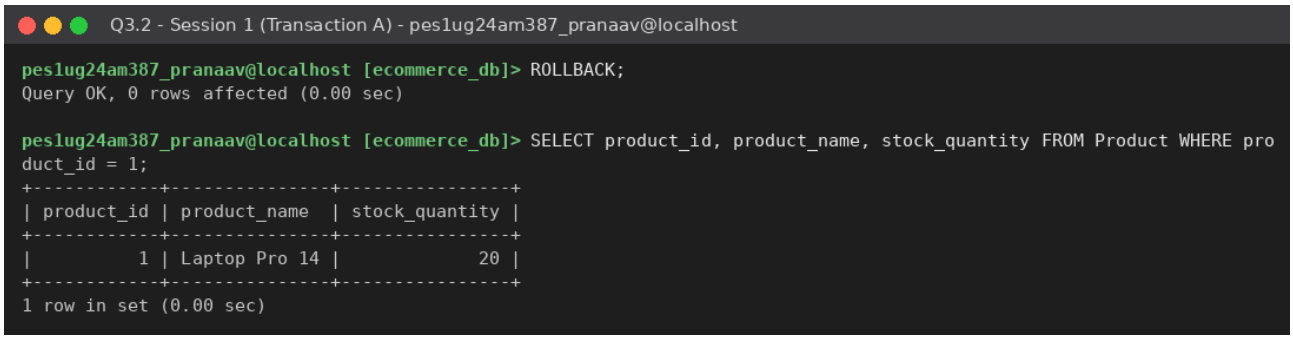


Screenshot: Question 3.2 - Session 2 (Transaction B)

Question 3.2 (continued) - rollback of Transaction A

SQL executed - Session 1 (Transaction A):

```
ROLLBACK;
SELECT product_id, product_name, stock_quantity FROM Product WHERE product_id = 1;
```



Screenshot: Question 3.2 - Session 1 (Transaction A)

Observation: Yes. Transaction B read stock_quantity = 5, which is A's uncommitted change, so this is a **dirty read**. After both transactions rolled back, the stock went back to 20. The value B read was never committed.

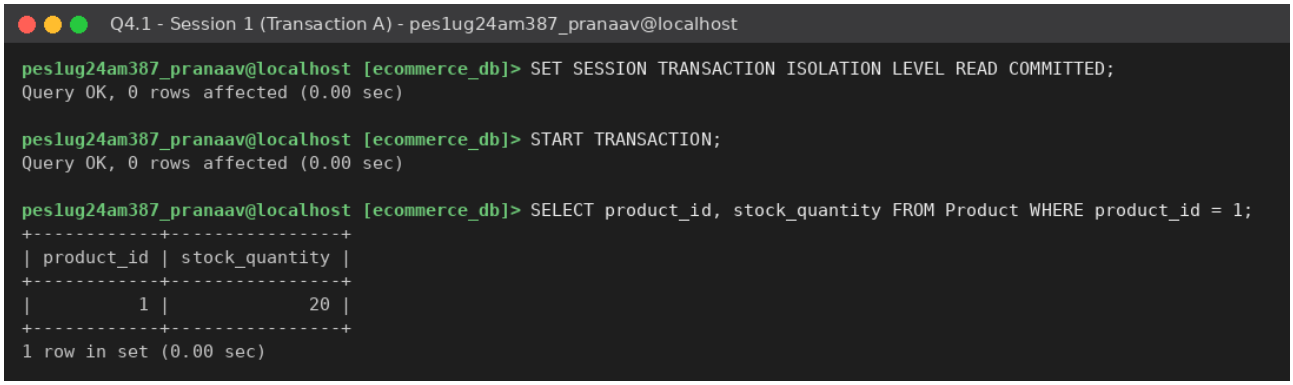
Task 4: READ COMMITTED

Question 4.1

In Session 1, set READ COMMITTED, start a transaction and read the stock_quantity of product 1.

SQL executed - Session 1 (Transaction A):

```
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
START TRANSACTION;
SELECT product_id, stock_quantity FROM Product WHERE product_id = 1;
```



Screenshot: Question 4.1 - Session 1 (Transaction A)

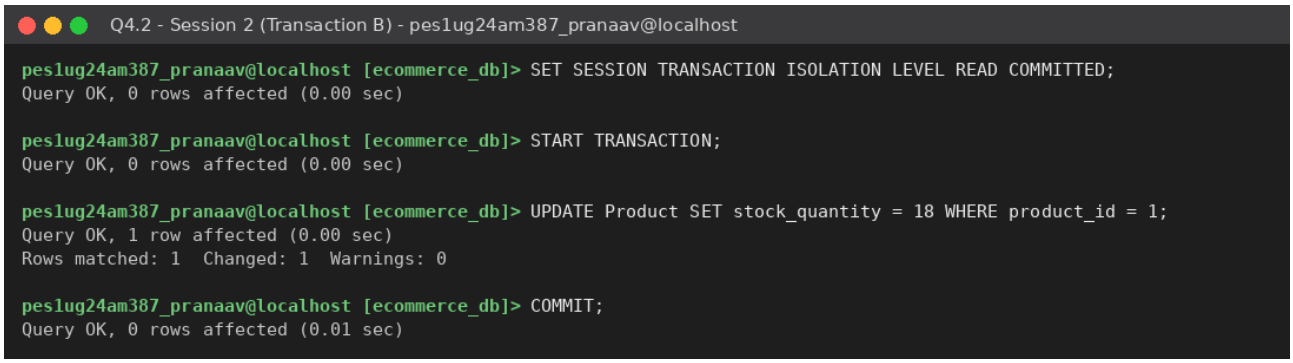
Observation: Transaction A reads stock_quantity = 20.

Question 4.2

In Session 2, set READ COMMITTED, start a transaction, change the stock_quantity of product 1 to 18 and commit.

SQL executed - Session 2 (Transaction B):

```
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
START TRANSACTION;
UPDATE Product SET stock_quantity = 18 WHERE product_id = 1;
COMMIT;
```



Screenshot: Question 4.2 - Session 2 (Transaction B)

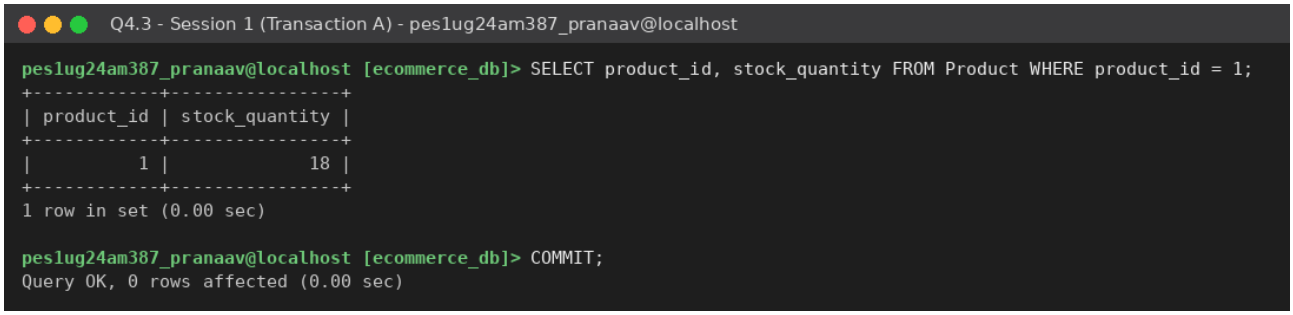
Observation: Transaction B set the stock to 18 and committed.

Question 4.3

In Session 1, without ending the transaction, read the stock_quantity of product 1 again and record the value. Then commit Transaction A.

SQL executed - Session 1 (Transaction A):

```
SELECT product_id, stock_quantity FROM Product WHERE product_id = 1;
COMMIT;
```



Screenshot: Question 4.3 - Session 1 (Transaction A)

Observation: Inside the same transaction, A now reads **18** instead of 20. READ COMMITTED shows each statement the latest committed data, so this is a non-repeatable read.

Task 5: REPEATABLE READ

Question 5.1

In Session 1, set REPEATABLE READ, start a transaction and display the product_id, product_name, price and stock_quantity of all active Electronics products.

SQL executed - Session 1 (Transaction A):

```
SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;
START TRANSACTION;
SELECT product_id, product_name, price, stock_quantity FROM Product WHERE category = 'Electronics' AND status =
    'Active';
```

Q5.1 - Session 1 (Transaction A) - pes1ug24am387_pranaav@localhost

```
pes1ug24am387_pranaav@localhost [ecommerce_db]> SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;
Query OK, 0 rows affected (0.00 sec)

pes1ug24am387_pranaav@localhost [ecommerce_db]> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

pes1ug24am387_pranaav@localhost [ecommerce_db]> SELECT product_id, product_name, price, stock_quantity FROM Product WH
ERE category = 'Electronics' AND status = 'Active';
+------------+--------------------+----------+----------------+
| product_id | product_name       | price    | stock_quantity |
+------------+--------------------+----------+----------------+
|          1 | Laptop Pro 14      | 85000.00 |             18 |
|          2 | Wireless Headphones |  6500.00 |             35 |
|          4 | Mechanical Keyboard |  4500.00 |             25 |
+------------+--------------------+----------+----------------+
3 rows in set (0.00 sec)
```

Screenshot: Question 5.1 - Session 1 (Transaction A)

Observation: Three active Electronics products are listed. Product 2 costs 6500.00.

Question 5.2

In Session 2, set REPEATABLE READ, start a transaction, change the price of product 2 to 7000.00 and commit.

SQL executed - Session 2 (Transaction B):

```
SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;
START TRANSACTION;
UPDATE Product SET price = 7000.00 WHERE product_id = 2;
COMMIT;
```

Q5.2 - Session 2 (Transaction B) - pes1ug24am387_pranaav@localhost

```
pes1ug24am387_pranaav@localhost [ecommerce_db]> SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;
Query OK, 0 rows affected (0.00 sec)

pes1ug24am387_pranaav@localhost [ecommerce_db]> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

pes1ug24am387_pranaav@localhost [ecommerce_db]> UPDATE Product SET price = 7000.00 WHERE product_id = 2;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

pes1ug24am387_pranaav@localhost [ecommerce_db]> COMMIT;
Query OK, 0 rows affected (0.00 sec)
```

Screenshot: Question 5.2 - Session 2 (Transaction B)

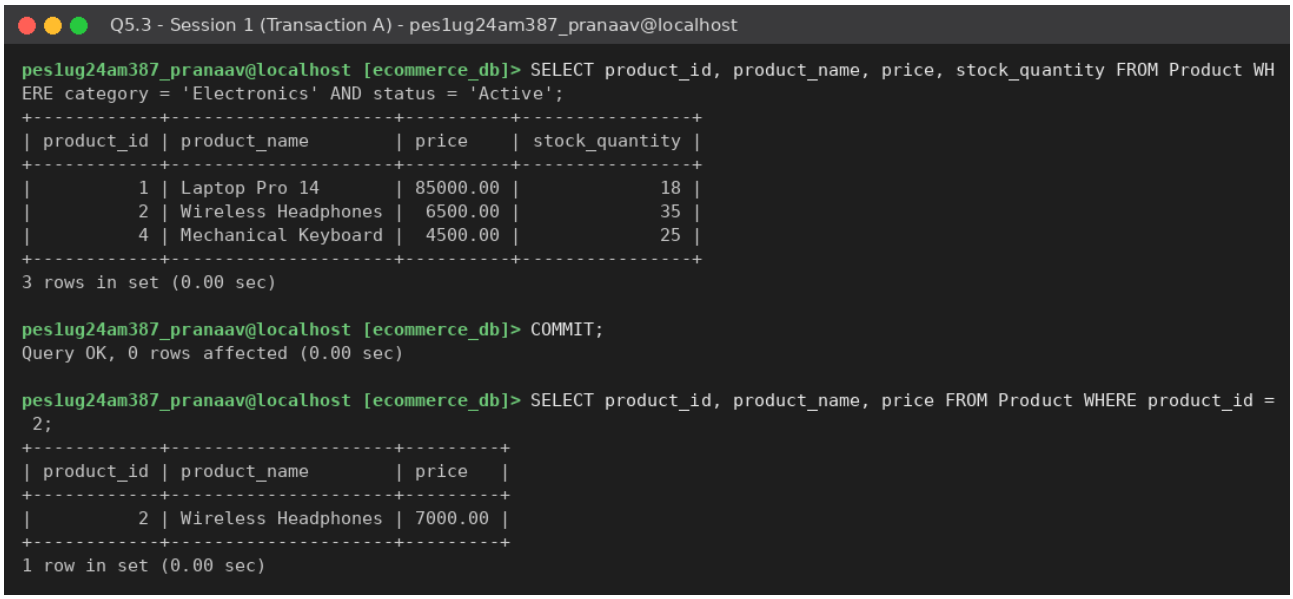
Observation: Transaction B changed product 2's price to 7000.00 and committed.

Question 5.3

In Session 1, repeat exactly the same query within the same transaction and record the price shown for product 2. Then commit Transaction A.

SQL executed - Session 1 (Transaction A):

```
SELECT product_id, product_name, price, stock_quantity FROM Product WHERE category = 'Electronics' AND status =
    'Active';
COMMIT;
SELECT product_id, product_name, price FROM Product WHERE product_id = 2;
```



Screenshot: Question 5.3 - Session 1 (Transaction A)

Observation: Product 2 still shows **6500.00** because A keeps reading the snapshot taken at its first read (MVCC consistent read). Only after A commits does a new read show 7000.00.

Task 6: SERIALIZABLE

Question 6.1

In Session 1, set SERIALIZABLE, start a transaction and display the product_id, product_name, category and status of all active Electronics products. Keep Session 1 open.

SQL executed - Session 1 (Transaction A):

```
SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;
START TRANSACTION;
SELECT product_id, product_name, category, status FROM Product WHERE category = 'Electronics' AND status =
    'Active';
```

Q6.1 - Session 1 (Transaction A) - pes1ug24am387_pranaav@localhost

```
pes1ug24am387_pranaav@localhost [ecommerce_db]> SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;
Query OK, 0 rows affected (0.00 sec)

pes1ug24am387_pranaav@localhost [ecommerce_db]> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

pes1ug24am387_pranaav@localhost [ecommerce_db]> SELECT product_id, product_name, category, status FROM Product WHERE c
ategory = 'Electronics' AND status = 'Active';
+------------+--------------------+-------------+--------+
| product_id | product_name       | category    | status |
+------------+--------------------+-------------+--------+
|          1 | Laptop Pro 14      | Electronics | Active |
|          2 | Wireless Headphones | Electronics | Active |
|          4 | Mechanical Keyboard | Electronics | Active |
+------------+--------------------+-------------+--------+
3 rows in set (0.00 sec)
```

Screenshot: Question 6.1 - Session 1 (Transaction A)

Observation: Under SERIALIZABLE the plain SELECT takes shared next-key locks on the idx_category range for 'Electronics'.

Question 6.2

In Session 2, set SERIALIZABLE, start a transaction and insert a new active Electronics product, USB-C Dock (price 8000.00, stock 15), while Session 1 remains open. Observe what happens to the INSERT.

SQL executed - Session 2 (Transaction B):

```
SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;
START TRANSACTION;
INSERT INTO Product (product_name, category, price, stock_quantity, status) VALUES ('USB-C Dock', 'Electronics',
    8000.00, 15, 'Active');
```

Q6.2 - Session 2 (Transaction B) - pes1ug24am387_pranaav@localhost

```
pes1ug24am387_pranaav@localhost [ecommerce_db]> SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;
Query OK, 0 rows affected (0.00 sec)

pes1ug24am387_pranaav@localhost [ecommerce_db]> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

pes1ug24am387_pranaav@localhost [ecommerce_db]> INSERT INTO Product (product_name, category, price, stock_quantity, st
atus) VALUES ('USB-C Dock', 'Electronics', 8000.00, 15, 'Active');

   [ ... INSERT is waiting for lock - no prompt returned ... ]
```

Screenshot: Question 6.2 - Session 2 (Transaction B)

Observation: The INSERT **blocks**. It waits for an insert-intention lock on the range that Session 1 has locked, and no prompt comes back.

Question 6.3

Return to Session 1 and commit Transaction A.

SQL executed - Session 1 (Transaction A):

```
COMMIT;
```

Q6.3 - Session 1 (Transaction A) - pes1ug24am387_pranaav@localhost

```
pes1ug24am387_pranaav@localhost [ecommerce_db]> COMMIT;
Query OK, 0 rows affected (0.00 sec)
```

Screenshot: Question 6.3 - Session 1 (Transaction A)

Observation: Transaction A commits and releases its range locks.

Question 6.4

Return to Session 2, record what happened to the INSERT after Session 1 committed, and commit Transaction B.

SQL executed - Session 2 (Transaction B):

```
COMMIT;
SELECT product_id, product_name, category, status FROM Product WHERE category = 'Electronics' AND status =
    'Active';
```

Q6.4 - Session 2 (Transaction B) - pes1ug24am387_pranaav@localhost

```
   [ ... blocked INSERT from Q6.2 resumes after Session 1 COMMIT ... ]
Query OK, 1 row affected (4.55 sec)

pes1ug24am387_pranaav@localhost [ecommerce_db]> COMMIT;
Query OK, 0 rows affected (0.01 sec)

pes1ug24am387_pranaav@localhost [ecommerce_db]> SELECT product_id, product_name, category, status FROM Product WHERE c
ategory = 'Electronics' AND status = 'Active';
+------------+--------------------+-------------+--------+
| product_id | product_name       | category    | status |
+------------+--------------------+-------------+--------+
|          1 | Laptop Pro 14      | Electronics | Active |
|          2 | Wireless Headphones | Electronics | Active |
|          4 | Mechanical Keyboard | Electronics | Active |
|          5 | USB-C Dock         | Electronics | Active |
+------------+--------------------+-------------+--------+
4 rows in set (0.00 sec)
```

Screenshot: Question 6.4 - Session 2 (Transaction B)

Observation: The waiting INSERT finished right after A committed: 'Query OK, 1 row affected (4.55 sec)', and the time includes the wait. B then committed and USB-C Dock (product_id 5) is now in the table.

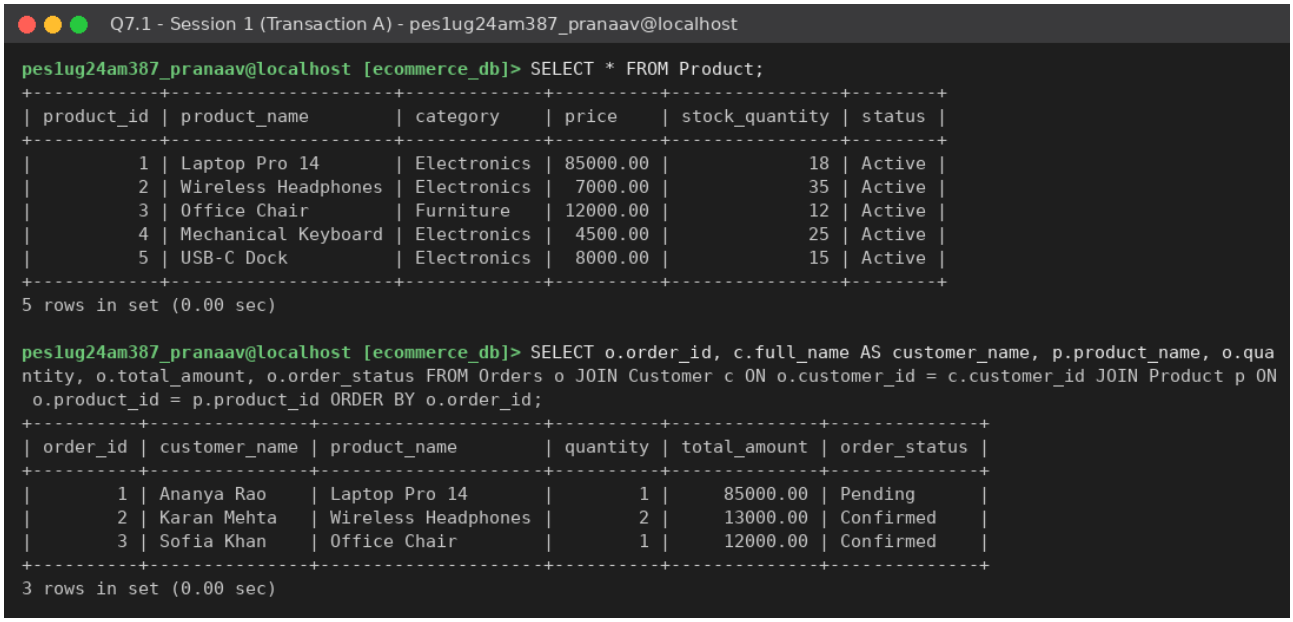
Task 7: Verification Queries

Question 7.1

Display all products, and display each order with its customer name, product name, quantity, total amount and order status by joining Orders, Customer and Product.

SQL executed - Session 1 (Transaction A):

```
SELECT * FROM Product;
SELECT o.order_id, c.full_name AS customer_name, p.product_name, o.quantity, o.total_amount, o.order_status FROM
    Orders o JOIN Customer c ON o.customer_id = c.customer_id JOIN Product p ON o.product_id = p.product_id
    ORDER BY o.order_id;
```



Screenshot: Question 7.1 - Session 1 (Transaction A)

Observation: Final state: product 1 stock = 18, product 2 price = 7000.00, USB-C Dock added as product 5. The 3 orders are unchanged.

Observation and Comparison

Isolation Level	Main Demonstration	Observation	Concurrency
READ UNCOMMITTED	Dirty read	Transaction B read stock = 5 for product 1 before Transaction A committed. A then rolled back to 20, so B had read data that was never committed (dirty read).	Highest concurrency
READ COMMITTED	Committed-value visibility	Transaction A read stock 20, then 18 in the same transaction after B committed. Only committed data is visible, but a repeated read can return a different value (non-repeatable read).	High
REPEATABLE READ	Consistent repeated read	Transaction A saw price 6500.00 for product 2 both times, even though B had committed 7000.00. The MVCC snapshot keeps reads within a transaction consistent. The new value showed only after A committed.	Moderate
SERIALIZABLE	Blocking concurrent range operation	B's INSERT of an Active Electronics product blocked on A's shared range (next-key) locks. It finished only after A committed (about 4.5 s wait), which prevents phantom rows.	Lowest concurrency

Conclusion: Moving from READ UNCOMMITTED to SERIALIZABLE removes more anomalies (dirty read, then non-repeatable read, then phantom) but costs concurrency. InnoDB's default, REPEATABLE READ, uses MVCC snapshots, so ordinary reads stay consistent without blocking writers.